COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/2/2026	**DATE SUBMITTED:**	10/6/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 10.1 **GRAPHS**

1. PROCEDURE OUTPUT

ILO A:

CODE

DIMAGUIBA_GRAPHS > main.cpp > ...

```cpp
#include <iostream>

//store adjacency items
struct adjNode{
    int val, cost;
    adjNode* next;
};

//structure that store edges
struct graphEdge{
    int startVer, endVer, weight;
};

class Diagraph{
    private:
        //insert new nodes
        adjNode* getAdjListNode(int value, int weight, adjNode* head){
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        // point new to current head
        newNode->next = head;
        return newNode;
    }
    //number of nodes in graph
    int N;

    public:
        adjNode** head;
        //constructor
        Diagraph (graphEdge edges[], int n, int N){
            //allocate
            head = new adjNode* [N]();
            this->N = N;

            //initialize head pointer for the verices
            for(unsigned i = 0; i < n; ++i){
                int start_ver = edges[i].startVer;
                int end_ver = edges[i].endVer;
                int weight = edges[i].weight;
                //insert to beginning
                adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

```

```
                //point head to new node
                head[start_ver] = newNode;
            }
        }

        //Destructor
        ~Diagraph(){
            for (int i = 0; i < N; i++){
                adjNode* curr = head[i];
                while(curr != nullptr){
                    adjNode* temp = curr;
                    curr = curr->next;
                    delete temp;
                }
            }
        };

        //print all adjacent vertices of the vertex
        void display_AdjList(adjNode* ptr, int i){
            while (ptr != nullptr){
                std::cout<< "(" << i << ", "<< ptr->val << ", "<< ptr->cost<<") ";
                ptr = ptr->next;
            }
            std::cout<<std::endl;
        }
    };

    //Graph Implementation
    int main(){
        //graph the edges array
        graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
            {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}};
        int N = 6;
        //calculate number of edges
        int n = sizeof(edges)/sizeof(edges[0]);
        //construct graph
        Diagraph diagraph(edges, n, N);
        //print adjacency list representation of graph
        std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight): "<<std::endl;
        for (int i = 0; i < N; i++){
            // display adjacent vertices of vertex i
            diagraph.display_AdjList(diagraph.head[i], i);
        }
        return 0;
    }
```

OUTPUT

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ILO B:

CODE

Step 1:

```
MAGUIBA_GRAPHS > DFS.cpp >
1   #include <string>
2   #include <vector>
3   #include <iostream>
4   #include <set>
5   #include <map>
6   #include <stack>
7
8   template <typename T>
9   class Graph;
```

Step 2:

```
template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    //compare edges by their weights, not the source/destination verts

    inline bool operator < (const Edge<T> &e) const{
        return this->weight < e.weight;
    }
    inline bool operator > (const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};
```

Step 3:

```
27  template <typename T>
28  std::ostream &operator<<(std::ostream &os, const Graph <T> &G){
29      for (auto i = 1; i < G.vertices(); i++){
30          os << i << ":\t";
31          auto edges = G.outgoing_edges(i);
32          for (auto &e : edges)
33              os <<"{"<< e.dest << ": "<< e.weight << "}, ";
34              os<<std::endl;
35      }
36      return os;
37  }
```

Step 4:

```cpp
template <typename T>
class Graph{
    public:
    //initialize graph with N verts
    Graph(size_t N) : V(N){}

    //return number of verts in graph
    auto vertices() const{
        return V;
    }
    //return all edges in graph
    auto &edges() const{
        return edge_list;
    }

    void add_edge(Edge<T> &&e){
        //check if source and destinations are within range
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
        edge_list.emplace_back(e);

        else
            std::cerr<<"Vertex out of bounds"<<std::endl;
    }
    //retrun all outgoing edges from vert v
    auto outgoing_edges(size_t v) const{
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list){
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    //overloads the << op so graph can be written to a stream directly
    //can be used as std::cout << obj <<std:::endl;

    template <typename U>
    friend std::ostream &operator <<(std::ostream &os, const Graph<U> &G);
    private:
        size_t V; //store nuimber of verts in graph
        std::vector<Edge<T>> edge_list;

};

```

Step 5:

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest){
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set <size_t> visited;
    stack.push(1); //Assume that DFS alwats starrs from vertex ID 1
    while (!stack.empty()){
        auto current_vertex = stack.top();
        stack.pop();
        //if current vert hassn't been visited before
        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex)){
                //if vert hasn't been visited, insert to stack
                if(visited.find(e.dest) == visited.end()){
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

Step 6-7:

```
106
107  template <typename T>
108  auto create_reference_graph(){
109      Graph<T> G(9);
110      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
111      edges[1] = {{2, 0}, {5, 0}};
112      edges[2] = {{1, 0}, {5, 0}, {4, 0}};
113      edges[3] = {{4, 0}, {7, 0}};
114      edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
115      edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
116      edges[6] = {{4, 0}, {7, 0}, {8, 0}};
117      edges[7] = {{3, 0}, {6, 0}};
118      edges[8] = {{4, 0}, {5, 0}, {6, 0}};
119      for (auto &i : edges)
120          for(auto &j : i.second)
121              G.add_edge(Edge<T>{i.first, j.first, j.second});
122              return G;
123  }
124
125  template <typename T>
126  void test_DFS(){
127      //create an instance and print the graph
128      auto G = create_reference_graph<unsigned>();
129      std::cout << G << std::endl;
130      //run DFS starting from vertex ID 1 and print visit order
131      std::cout<< "DFS Order of vertices: "<< std::endl;
132      auto dfs_visit_order = depth_first_search(G, 1);
133      for (auto v : dfs_visit_order)
134          std::cout<< v<< std::endl;
135  }
136
137  int main(){
138      using T = unsigned;
139      test_DFS<T>();
140      return 0;
141  }
142
```

OUTPUT

```
PS C:\Users\shosh\Documents\VSCODE>  ${env:DEBUGIN
crosoft-MIEngine-Pid-g5bypu3c.qqh' '--dbgExe=C:\ms
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\shosh\Documents\VSCODE>
```

ILO B2:

CODE

Step 1:

```
DIMAGUIBA_GRAPHS > BFS.cpp > Edge
1   #include <string>
2   #include <vector>
3   #include <iostream>
4   #include <set>
5   #include <map>
6   #include <queue>
7
8   template <typename T>
9   class Graph;
```

Step 2:

```
10
11  template <typename T>
12  struct Edge{
13      size_t src;
14      size_t dest;
15      T weight;
16
17      inline bool operator <(const Edge<T> &e) const{
18          return this->weight < e.weight;
19      }
20
21      inline bool operator > (const Edge<T> &e) const{
22          return this->weight > e.weight;
23      }
24  };
25
```

Step 3:

```
26  template <typename T>
27  std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
28      for(auto i = 1; i < G.vertices(); i++){
29          os << i << ":\t";
30
31          auto edges = G.outgoing_edges(i);
32          for(auto &e : edges)
33              os << "{" << e.dest << ": "<< e.weight << "}, ";
34          os << std::endl;
35      }
36      return os; //will not work unless returns something (os works)
37  }
38
```

Step 4:

```cpp
template <typename T>
class Graph{
    public:
    Graph(size_t N) : V(N){}

    auto vertices() const{
        return V;
    }

    auto &edges() const{
        return edge_list;
    }

    void add_edge(Edge<T> &&e){
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
        edge_list.emplace_back(e);
        else
            std::cerr<< "Vertex out of bounds"<< std::endl;
    }

    auto outgoing_edges(size_t v) const{
        std::vector<Edge<T>> edges_from_v;
        for(auto &e : edge_list){
            if (e.src == v){
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

    private:
        size_t V;
        std::vector<Edge<T>> edge_list;
};

```

Step 5:

```cpp
template <typename T>
auto create_reference_graph(){
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T >>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for(auto &i : edges)
        for(auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
            return G;
}

```

Step 6:

```
 96  template <typename T>
 97  auto breadth_first_search (const Graph<T> &G, size_t dest){
 98      std::queue<size_t> queue;
 99      std::vector<size_t> visit_order;
100      std::set<size_t> visited;
101      queue.push(1);//assume BFS always starts at vert ID 1
102      while(!queue.empty()){
103          auto current_vertex = queue.front();
104          queue.pop();
105          //if current vert hasn't been visited
106          if(visited.find(current_vertex) == visited.end()){
107              visited.insert(current_vertex);
108              visit_order.push_back(current_vertex);
109              for(auto e : G.outgoing_edges(current_vertex))
110                  queue.push(e.dest);
111          }
112      }
113      return visit_order;
114  }
115
116
```

Step 7:

```
117  template <typename T>
118  void test_BFS(){
119      //create an instance and print the graph
120      auto G = create_reference_graph<unsigned>();
121      std::cout<< G<< std::endl;
122      auto bfs_visit_order = breadth_first_search(G,1);
123      for (auto v : bfs_visit_order)
124          std::cout << v << std::endl;
125  }
126
127  int main(){
128      using T = unsigned;
129      test_BFS<T>();
130      return 0;
131  }
132
```

OUTPUT

```
PS C:\Users\shosh\Documents\VSCODE>  ${env:DEBUGINF
32-x64\debugAdapters\bin\WindowsDebugLauncher.exe'
na1dbxg.x0w' '--stderr=Microsoft-MIEngine-Error-bst
64\bin\gdb.exe' '--interpreter=mi'
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

1
2
5
4
8
3
6
7
PS C:\Users\shosh\Documents\VSCODE>
```

Observation/Analysis:

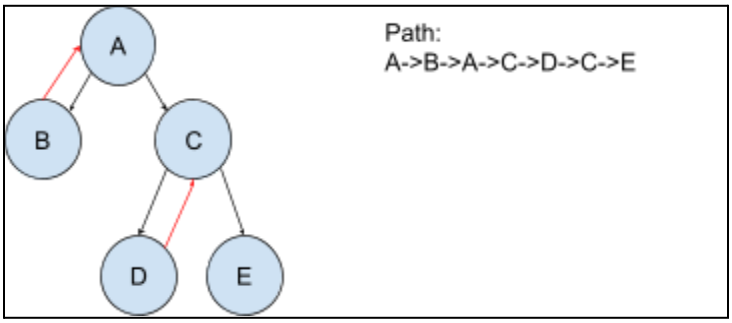
Both algorithms are quite similar with each other in terms of structure, but their methods are where they differ. DFS uses backtracking, where once there aren't any more nodes to move to, they move back or push back in order to search for more nodes or vertices or when all vertices are visited, using stacks in where it takes the items and pushes them into a stack of those visited. BFS does something similar in where it traverses from a selected node, but instead, goes from neighbor to neighbor to traverse till it reaches the last layer. This is done through queues instead, doing the same thing of queuing into a list of those visited and putting the unvisited ones in the back this time. These algorithms are evident as looking at the code shows them using the same libraries.

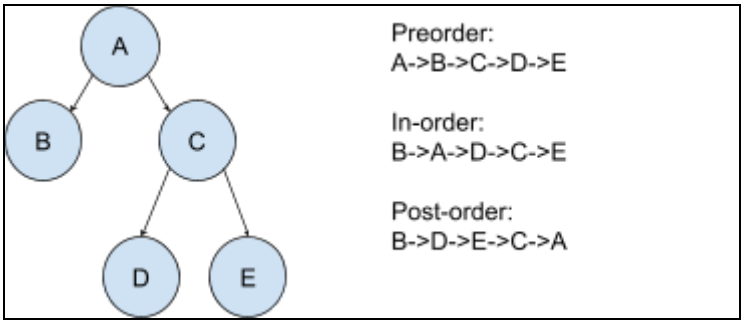
They were also weighted, that's why the Edge functions keep track of the source, its destination or location, and the weight of that edge depending on the given. The weight is also compared with each other and returns a boolean value (T/F) when the weights of the edges are greater or less than each other. Both functions have this. The Graph function in the different algorithms initialize with a set number of vertices. It then returns all those edges. Outside of the traversal and stack and queue implementations, their graph constructions are relatively the same, with the given destinations and weight assigned to each edge, including those adjacent.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.
The best option would definitely be the DFS algorithm. Since the person would have to visit a location more than once through backtracking, returning to a visited location to explore other places after moving back. The overall system in which DFS works aligns with that person's goal.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.
It is very reminiscent of Binary Trees from before. Mainly the ordered traversals (Pre, In, and Post order). Comparing the 2, since you are able to pick or select a node to begin with to which you may back-track, the overall path can be different.





3. In the performed code, what data structure is used to implement the Breadth First Search?
The data structure used for BFS was the queue data structure.

4. How many times can a node be visited in the BFS?
Only once. This is evident with code tracking which vertices or nodes have already been visited.

Analysis of Supplementary + Lessons:
The new algorithms share familiar traits with trees, the only difference being the method in which they traverse each node or vertex. There is also the freedom (especially in DFS) to select which vertex to begin with and return to when needed. The data structures used differ from the type. DFS uses stacks while BFS uses queues. DFS traverses and allows backtracking to reach other vertices; BFS only visits each node or vertex once. In programming, they are split into different components, like Edges, Graph, and the searching itself.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- I am going to be very honest, I barely understand most of the procedure code. New libraries being introduced that I am not familiar with using, new keywords that popped up, and concepts that seem new to me. The algorithms aren't that difficult to understand, but when it comes to application through code, I do feel quite lost in some parts. I would say that I'm not that confident yet.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **After following the procedure, I found myself kind of lost with new concepts and libraries being used in the code. I needed something that could explain it to me and I can ask questions with. It helped quite a bit.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.